PYTHON ACTIVITY BASED LEARNING

CASE STUDIES:

1.Examination Result and Grace-Mark Validation

2.Attendance Recovery Computation

3. ATM Transaction Sequence Simulation

4. Fuel Consumption and Route Comparison

5. Progressive Water Tariff Analysis

6.Advanced Numeric Pattern Generator

7. Password Policy Validation Engine

8. String-Based Examination Answer Evaluation

9. Student Feedback Sentiment Scoring

10. Text Compression Using Character Runs

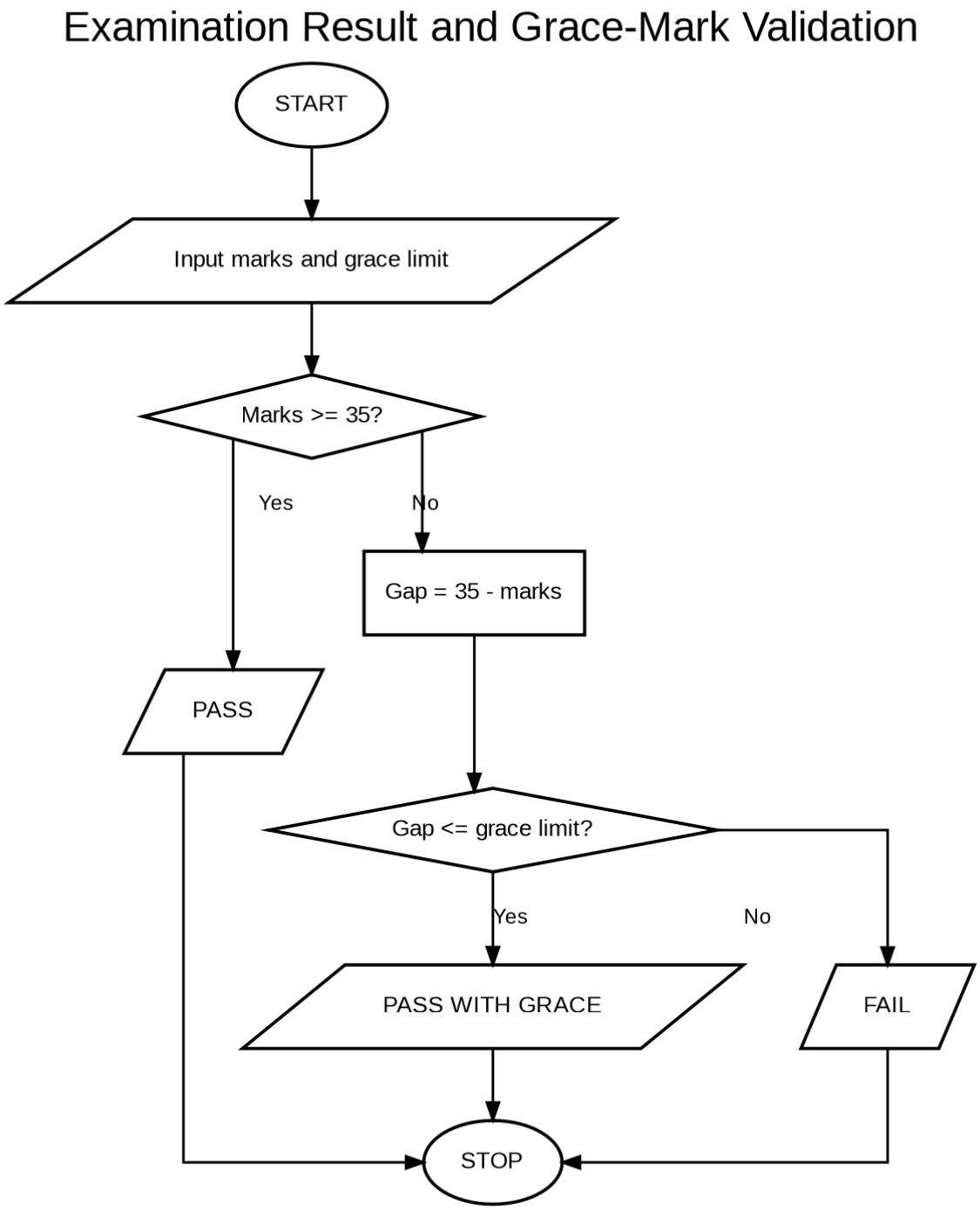
PROGRAM 1: EXAMINATION RESULT AND GRACE MARKS VALIDATION

1. Algorithm

1. Start.
2. Read the student's marks.
3. Read the grace marks limit.
4. Check whether the marks are between 0 and 100.
5. If the marks are invalid or the grace marks limit is negative, display "Invalid Input".
6. If the marks are greater than or equal to 35, display "Pass".
7. Otherwise, calculate the difference between 35 and the student's marks.
8. If the difference is less than or equal to the grace marks limit, display "Pass with Grace Marks".

9. Otherwise, display "Fail".
10. Stop.

2. Flowchart



3. Pseudocode

BEGIN

```
    INPUT marks
    INPUT grace

    IF marks < 0 OR marks > 100 OR grace < 0 THEN
        PRINT "Invalid Input"

    ELSE IF marks >= 35 THEN
        PRINT "Pass"

    ELSE IF 35 - marks <= grace THEN
        PRINT "Pass with Grace Marks"

    ELSE
        PRINT "Fail"

    END IF

END
```

4. Python Code

```
marks = int(input("Enter marks: "))
grace = int(input("Enter grace marks limit: "))

if marks < 0 or marks > 100 or grace < 0:
    print("Invalid Input")
```

```
elif marks >= 35:
   print("Pass")
elif 35 - marks <= grace:
   print("Pass with Grace Marks")
else:
   print("Fail")
```

5. Sample Output

Enter marks: 32

Enter grace marks limit: 3

Pass with Grace Marks

6. Test Cases

Test Case	Marks	Grace Limit	Expected Output
TC1	40	3	Pass
TC2	32	2	Fail
TC3	32	3	Pass with Grace Marks
TC4	35	0	Pass
TC5	-1	3	Invalid Input
TC6	101	3	Invalid Input
TC7	30	-2	Invalid Input

7. Result

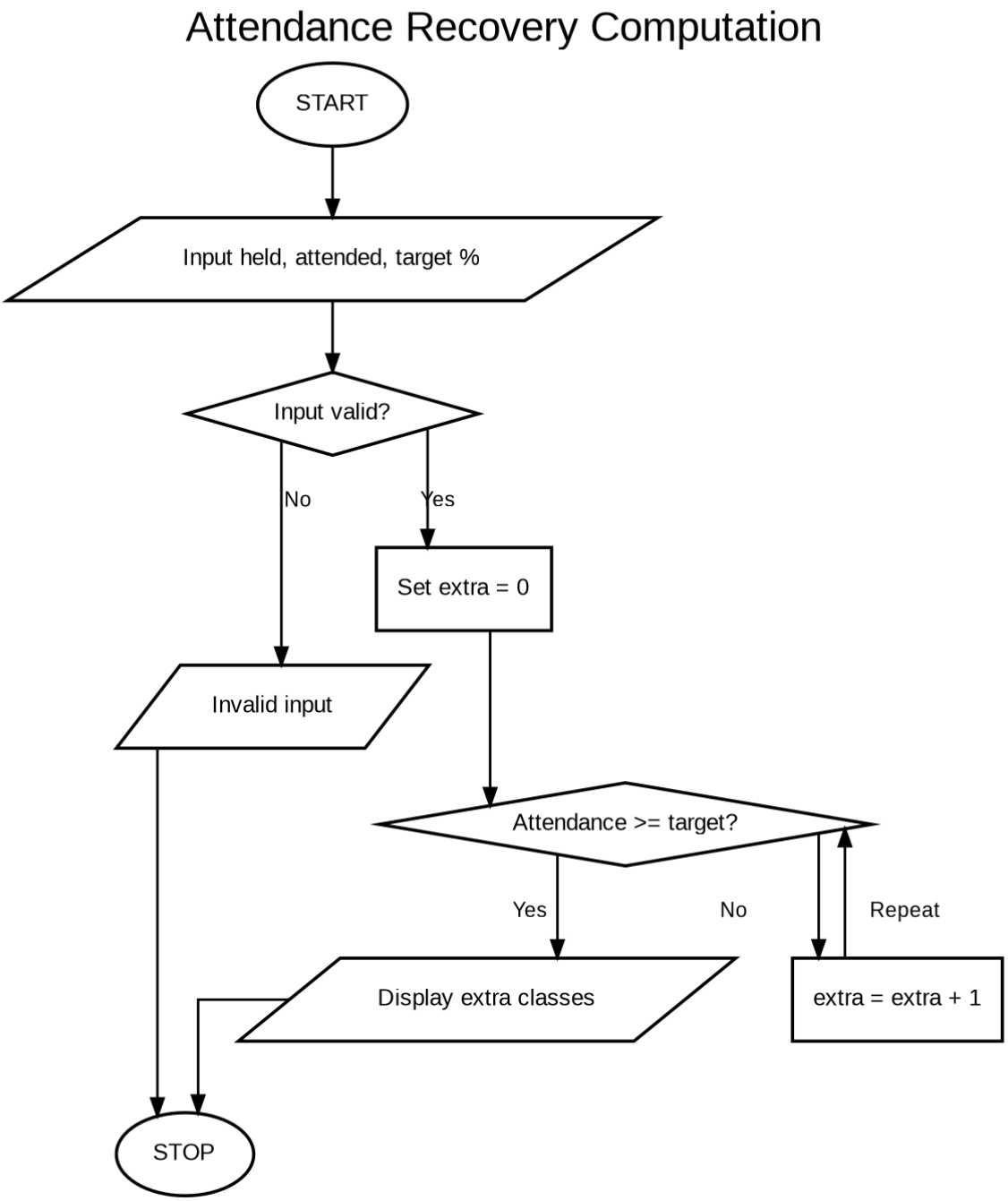
The program successfully determines the examination result based on the student's marks and the grace marks limit.

PROGRAM 2: ATTENDANCE RECOVERY CALCULATOR

1. Algorithm

1. Start.
2. Read the total number of classes held.
3. Read the number of classes attended.
4. Read the required attendance percentage.
5. Check whether the input values are valid.
6. If the input values are invalid, display "Invalid Input".
7. Initialize the number of additional classes to zero.
8. Calculate whether the required attendance percentage has been reached.
9. If the target is not reached, increase the number of additional classes by one.
10. Repeat the calculation until the target attendance percentage is reached.
11. If no additional classes are required, display "Target Achieved".
12. Otherwise, display the number of additional classes required.
13. Stop.

2. Flowchart



3. Pseudocode

BEGIN

INPUT held

INPUT attended

```
  INPUT target

  IF held < 0 OR attended < 0 OR attended > held THEN
    PRINT "Invalid Input"

  ELSE IF target <= 0 OR target > 100 THEN
    PRINT "Invalid Target"

  ELSE
    extra = 0

    WHILE (attended + extra) * 100 <
        target * (held + extra)
      extra = extra + 1
    END WHILE

    IF extra == 0 THEN
      PRINT "Target Achieved"
    ELSE
      PRINT "Additional classes required:", extra
    END IF

  END IF

END
```

4. Python Code

```
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
target = float(input("Enter target percentage: "))

if held < 0 or attended < 0 or attended > held:
    print("Invalid Input")
elif target <= 0 or target > 100:
    print("Invalid Target")
else:
    extra = 0

    while (attended + extra) * 100 < target * (held + extra):
        extra += 1

    if extra == 0:
        print("Target Achieved")
    else:
        print("Additional classes required:", extra)
```

5. Sample Output

Enter total classes held: 80

Enter classes attended: 56

Enter target percentage: 75

Additional classes required: 16

6. Test Cases

Test Case	Classes Held	Classes Attended	Target (%)	Expected Output
TC1	80	56	75	Additional classes required: 16
TC2	50	40	75	Target Achieved
TC3	10	2	75	Additional classes required: 22
TC4	10	10	75	Target Achieved
TC5	10	12	75	Invalid Input
TC6	10	5	0	Invalid Target
TC7	10	5	101	Invalid Target

7. Result

The program successfully calculates the number of additional classes required to achieve the target attendance percentage.

PROGRAM 3: ATM TRANSACTION SIMULATOR

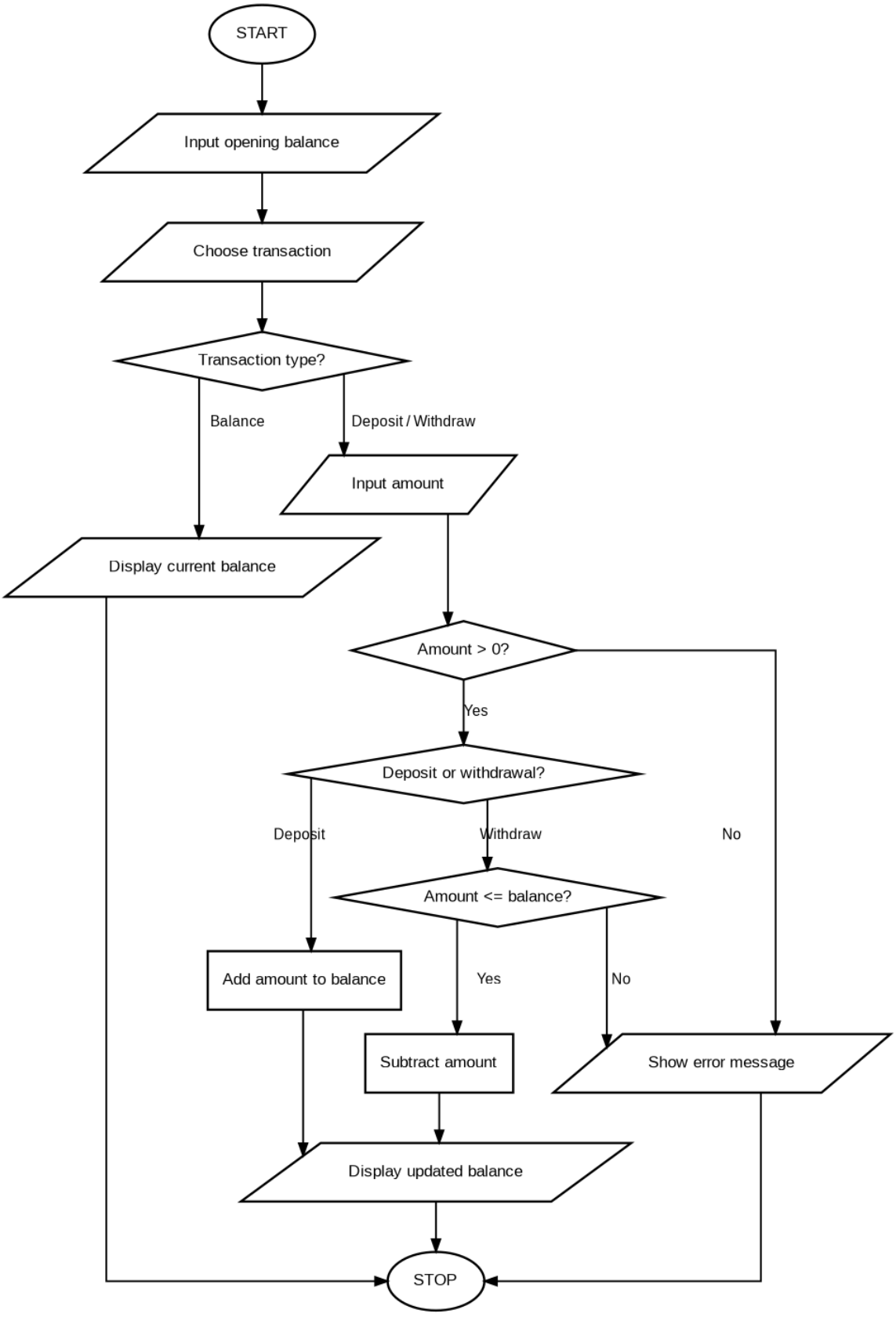
1. Algorithm

1. Start.
2. Read the initial account balance.
3. Check whether the balance is valid.
4. If the balance is negative, display "Invalid Balance".
5. Otherwise, display the transaction options: Balance, Deposit, and Withdraw.
6. Read the user's choice.
7. If the choice is Balance, display the current balance.
8. If the choice is Deposit, read the deposit amount.

9. If the deposit amount is positive, add it to the balance and display the updated balance.
10. If the choice is Withdraw, read the withdrawal amount.
11. If the withdrawal amount is invalid, display "Invalid Amount".
12. If the withdrawal amount is greater than the balance, display "Insufficient Balance".
13. Otherwise, subtract the withdrawal amount from the balance and display the remaining balance.
14. If the user enters an invalid choice, display "Invalid Choice".
15. Stop.

2. Flowchart

ATM Transaction Sequence Simulation



3. Pseudocode

```
BEGIN

  INPUT balance
  INPUT choice

  IF balance < 0 THEN
    PRINT "Invalid Balance"

  ELSE IF choice == "balance" THEN
    PRINT "Current Balance:", balance

  ELSE IF choice == "deposit" THEN
    INPUT amount

    IF amount > 0 THEN
      balance = balance + amount
      PRINT "Updated Balance:", balance
    ELSE
      PRINT "Invalid Amount"
    END IF

  ELSE IF choice == "withdraw" THEN
    INPUT amount
```

```
    IF amount <= 0 THEN
      PRINT "Invalid Amount"
    ELSE IF amount <= balance THEN
      balance = balance - amount
      PRINT "Remaining Balance:", balance
    ELSE
      PRINT "Insufficient Balance"
    END IF

  ELSE
    PRINT "Invalid Choice"
  END IF

END
```

4. Python Code

```
balance = float(input("Enter initial balance: "))
choice = input("Enter choice (balance/deposit/withdraw): ").lower()

if balance < 0:
  print("Invalid Balance")
elif choice == "balance":
  print("Current Balance:", balance)
elif choice == "deposit":
  amount = float(input("Enter deposit amount: "))
  if amount > 0:
```

```
        balance += amount
        print("Updated Balance:", balance)
    else:
        print("Invalid Amount")
elif choice == "withdraw":
    amount = float(input("Enter withdrawal amount: "))
    if amount <= 0:
        print("Invalid Amount")
    elif amount <= balance:
        balance -= amount
        print("Remaining Balance:", balance)
    else:
        print("Insufficient Balance")
else:
    print("Invalid Choice")
```

5. Sample Output

Enter initial balance: 1000

Enter choice (balance/deposit/withdraw): withdraw

Enter withdrawal amount: 250

Remaining Balance: 750.0

6. Test Cases

Test Case	Initial Balance	Transaction	Amount	Expected Output
TC1	1000	balance	—	Current Balance: 1000.0
TC2	1000	deposit	500	Updated Balance: 1500.0

TC3	1000	withdraw	250	Remaining Balance: 750.0
TC4	1000	withdraw	1500	Insufficient Balance
TC5	1000	deposit	-100	Invalid Amount
TC6	-100	balance	—	Invalid Balance
TC7	1000	transfer	—	Invalid Choice

7. Result

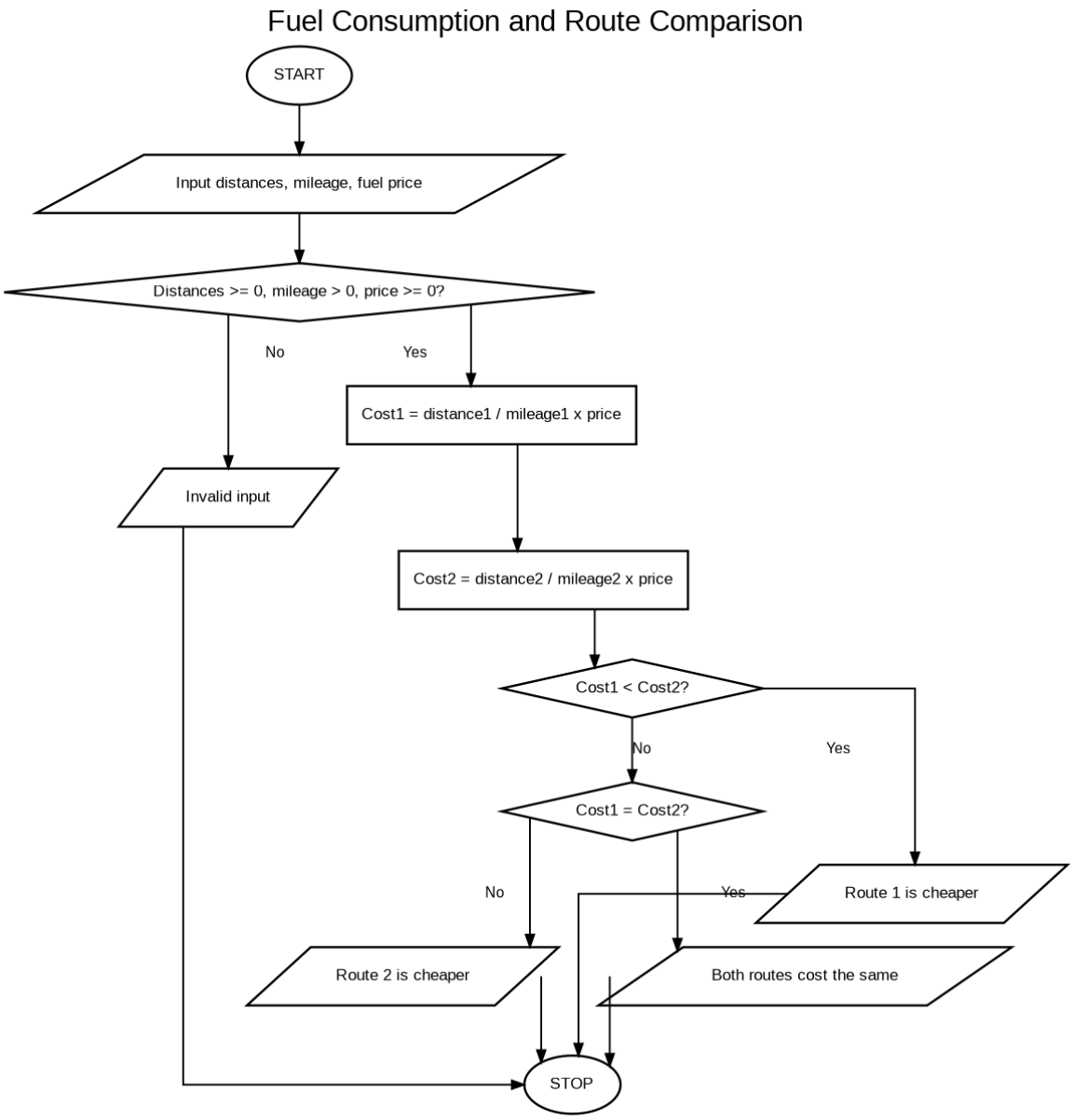
The program successfully simulates ATM transactions, including balance checking, depositing money, and withdrawing money.

PROGRAM 4: FUEL COST COMPARISON

1. Algorithm

1. Start.
2. Read the distance and mileage of Route 1.
3. Read the distance and mileage of Route 2.
4. Read the fuel price per litre.
5. Check whether the input values are valid.
6. If any distance is negative, any mileage is zero or negative, or the fuel price is negative, display "Invalid Input".
7. Calculate the fuel cost of Route 1 by dividing its distance by its mileage and multiplying by the fuel price.
8. Calculate the fuel cost of Route 2 using the same formula.
9. Display the fuel cost of both routes.
10. If Route 1 costs less, display "Route 1 is cheaper".
11. If Route 2 costs less, display "Route 2 is cheaper".
12. Otherwise, display "Both routes have equal cost".
13. Stop.

2. Flowchart



3. Pseudocode

BEGIN

INPUT distance1, mileage1

INPUT distance2, mileage2

INPUT fuel_price

```
IF distance1 < 0 OR distance2 < 0 OR
  mileage1 <= 0 OR mileage2 <= 0 OR
  fuel_price < 0 THEN

   PRINT "Invalid Input"

ELSE
   cost1 = (distance1 / mileage1) * fuel_price
   cost2 = (distance2 / mileage2) * fuel_price

   PRINT "Route 1 Fuel Cost:", cost1
   PRINT "Route 2 Fuel Cost:", cost2

   IF cost1 < cost2 THEN
      PRINT "Route 1 is cheaper"

   ELSE IF cost2 < cost1 THEN
      PRINT "Route 2 is cheaper"

   ELSE
      PRINT "Both routes have equal cost"
   END IF

END IF
```

END

4. Python Code

```
distance1 = float(input("Enter Route 1 distance: "))
mileage1 = float(input("Enter Route 1 mileage: "))
distance2 = float(input("Enter Route 2 distance: "))
mileage2 = float(input("Enter Route 2 mileage: "))
fuel_price = float(input("Enter fuel price per litre: "))

if (distance1 < 0 or distance2 < 0 or
    mileage1 <= 0 or mileage2 <= 0 or fuel_price < 0):
  print("Invalid Input")
else:
  cost1 = (distance1 / mileage1) * fuel_price
  cost2 = (distance2 / mileage2) * fuel_price

  print("Route 1 Fuel Cost:", cost1)
  print("Route 2 Fuel Cost:", cost2)

  if cost1 < cost2:
    print("Route 1 is cheaper")
  elif cost2 < cost1:
    print("Route 2 is cheaper")
  else:
    print("Both routes have equal cost")
```

5. Sample Output

Enter Route 1 distance: 100

Enter Route 1 mileage: 20

Enter Route 2 distance: 120

Enter Route 2 mileage: 15

Enter fuel price per litre: 100

Route 1 Fuel Cost: 500.0

Route 2 Fuel Cost: 800.0

Route 1 is cheaper

6. Test Cases

Test Case	Route 1 Distance	Route 1 Mileage	Route 2 Distance	Route 2 Mileage	Fuel Price	Expected Output
TC1	100	20	120	15	100	Route 1 is cheaper
TC2	50	10	50	10	100	Both routes have equal cost
TC3	100	10	50	10	100	Route 2 is cheaper
TC4	100	0	50	10	100	Invalid Input
TC5	-10	10	50	10	100	Invalid Input
TC6	100	20	120	15	-5	Invalid Input

7. Result

The program successfully calculates and compares the fuel costs of two routes and identifies the cheaper route.

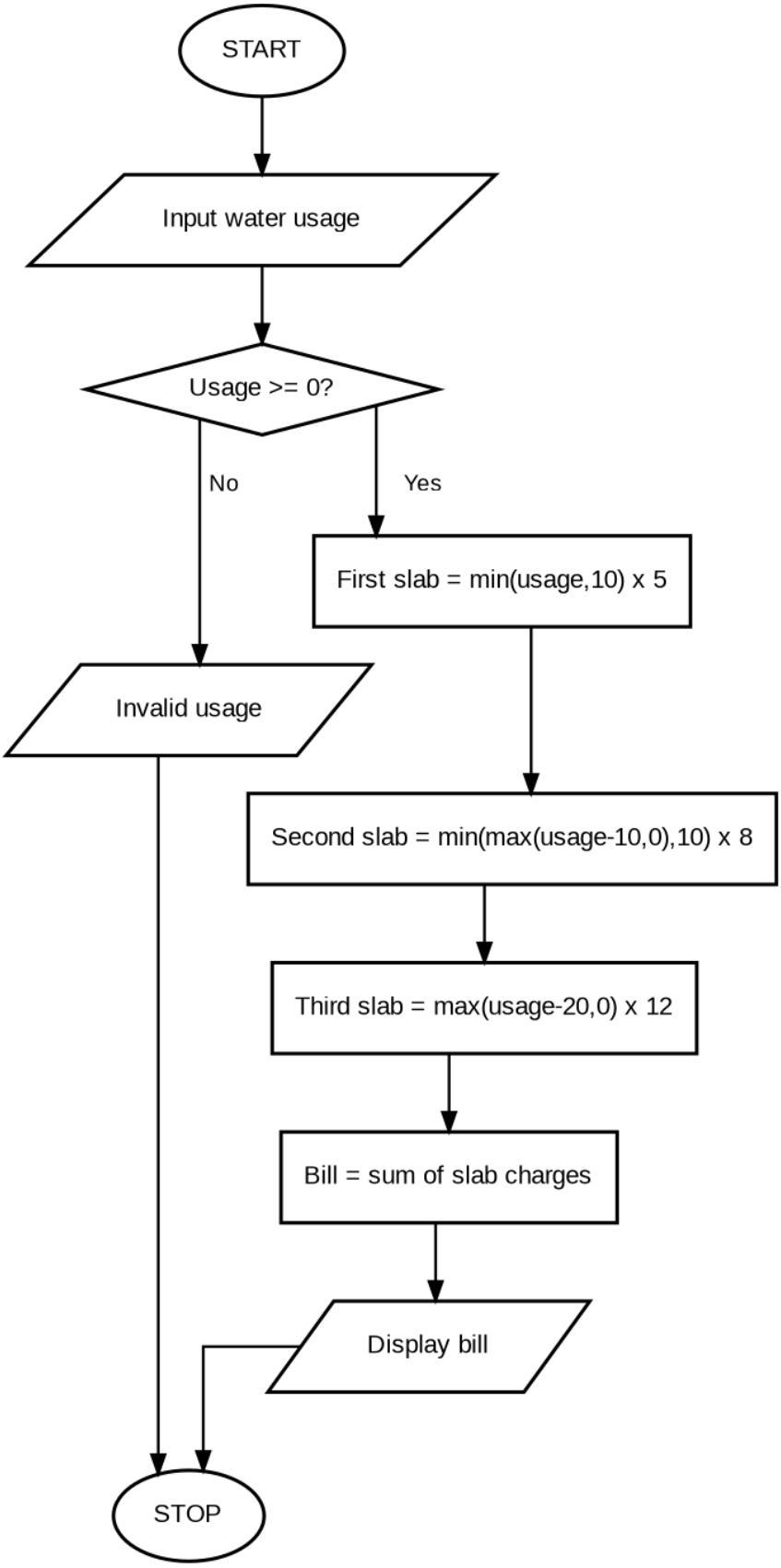
PROGRAM 5: WATER TARIFF CALCULATOR

1. Algorithm

1. Start.
2. Read the water consumption in units.
3. Check whether the consumption is negative.
4. If the consumption is negative, display "Invalid Consumption".
5. If the consumption is less than or equal to 10 units, calculate the bill at ₹5 per unit.
6. If the consumption is greater than 10 but less than or equal to 20 units, charge ₹5 per unit for the first 10 units and ₹8 per unit for the remaining units.
7. If the consumption is greater than 20 units, charge ₹5 per unit for the first 10 units, ₹8 per unit for the next 10 units, and ₹12 per unit for the remaining units.
8. Display the total water bill.
9. Stop.

2. Flowchart

Progressive Water Tariff Analysis



3. Pseudocode

```
BEGIN

  INPUT units

  IF units < 0 THEN
    PRINT "Invalid Consumption"

  ELSE IF units <= 10 THEN
    bill = units * 5
    PRINT "Total Water Bill:", bill

  ELSE IF units <= 20 THEN
    bill = (10 * 5) + ((units - 10) * 8)
    PRINT "Total Water Bill:", bill

  ELSE
    bill = (10 * 5) + (10 * 8) + ((units - 20) * 12)
    PRINT "Total Water Bill:", bill

  END IF

END
```

4. Python Code

```
units = float(input("Enter water consumption in units: "))
```

```
if units < 0:
    print("Invalid Consumption")
elif units <= 10:
    bill = units * 5
    print("Total Water Bill: Rs.", bill)
elif units <= 20:
    bill = (10 * 5) + ((units - 10) * 8)
    print("Total Water Bill: Rs.", bill)
else:
    bill = (10 * 5) + (10 * 8) + ((units - 20) * 12)
    print("Total Water Bill: Rs.", bill)
```

5. Sample Output

Enter water consumption in units: 25

Total Water Bill: Rs. 190.0

6. Test Cases

Tariff rates: First 10 units at ₹5 per unit, next 10 units at ₹8 per unit, and units above 20 at ₹12 per unit.

Test Case	Water Consumption (Units)	Expected Output
TC1	0	Total Water Bill: Rs. 0.0
TC2	10	Total Water Bill: Rs. 50.0
TC3	15	Total Water Bill: Rs. 90.0
TC4	20	Total Water Bill: Rs. 130.0
TC5	25	Total Water Bill: Rs. 190.0

TC6	-2	Invalid Consumption

7. Result

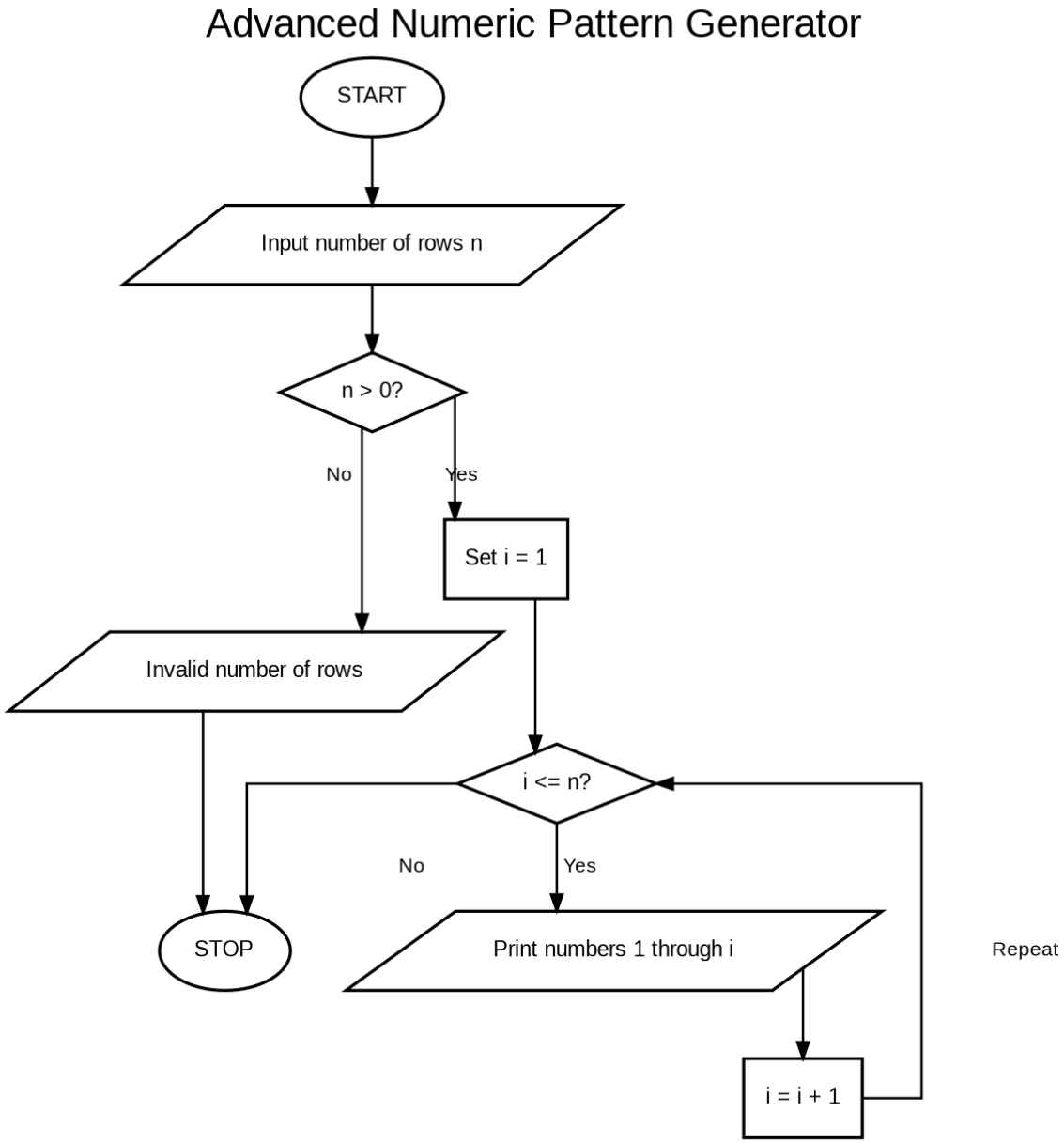
The program successfully calculates the total water bill based on the water consumption and the specified tariff rates.

PROGRAM 6: NUMERIC PATTERN PRINTING

1. Algorithm

1. Start.
2. Read the number of rows.
3. Check whether the number of rows is greater than zero.
4. If the number of rows is less than or equal to zero, display "Invalid Input".
5. Otherwise, initialize the outer loop from 1 to the number of rows.
6. For each row, initialize the inner loop from 1 to the current row number.
7. Print the current number without moving to a new line.
8. After printing all numbers in a row, move to the next line.
9. Repeat until all rows are printed.
10. Stop.

2. Flowchart



3. Pseudocode

```
BEGIN

  INPUT n

  IF n <= 0 THEN
    PRINT "Invalid Input"
```

```
  ELSE
    FOR i = 1 TO n
      FOR j = 1 TO i
        PRINT j WITHOUT NEW LINE
      END FOR

      PRINT NEW LINE
    END FOR

  END IF

END
```

4. Python Code

```
n = int(input("Enter number of rows: "))

if n <= 0:
  print("Invalid Input")
else:
  for i in range(1, n + 1):
    for j in range(1, i + 1):
      print(j, end="")
    print()
```

5. Sample Output

```
Enter number of rows: 4
```

1

12

123

1234

6. Test Cases

Test Case	Number of Rows	Expected Output
TC1	1	1
TC2	2	1, 12 (each on a separate line)
TC3	3	1, 12, 123 (each on a separate line)
TC4	4	1, 12, 123, 1234 (each on a separate line)
TC5	0	Invalid Input
TC6	-2	Invalid Input

7. Result

The program successfully prints a numeric pattern using nested loops based on the number of rows entered by the user.

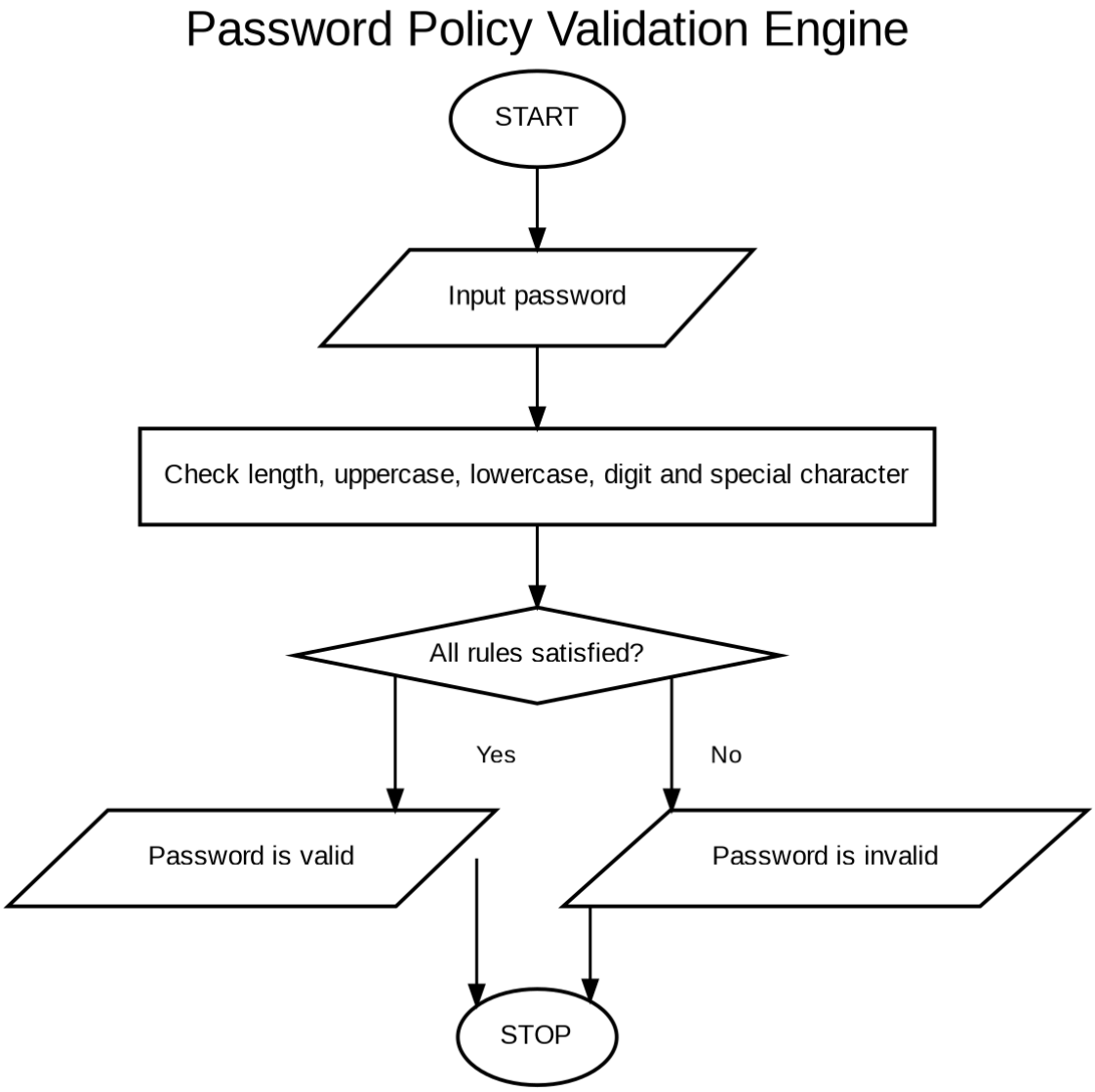
PROGRAM 7: PASSWORD STRENGTH VALIDATOR

1. Algorithm

1. Start.
2. Read the password from the user.
3. Check whether the password contains at least eight characters.
4. Check whether the password contains at least one uppercase letter.
5. Check whether the password contains at least one lowercase letter.
6. Check whether the password contains at least one digit.
7. Check whether the password contains at least one special character.

8. If all the conditions are satisfied, display "Strong Password".
9. Otherwise, display "Weak Password".
10. Stop.

2. Flowchart



3. Pseudocode

BEGIN

INPUT password

```
  IF LENGTH(password) < 8 THEN
    PRINT "Weak Password"

  ELSE IF password has no uppercase letter THEN
    PRINT "Weak Password"

  ELSE IF password has no lowercase letter THEN
    PRINT "Weak Password"

  ELSE IF password has no digit THEN
    PRINT "Weak Password"

  ELSE IF password has no special character THEN
    PRINT "Weak Password"

  ELSE
    PRINT "Strong Password"

  END IF

END
```

4. Python Code

```
import string
```

```
password = input("Enter password: ")

has_upper = any(ch.isupper() for ch in password)
has_lower = any(ch.islower() for ch in password)
has_digit = any(ch.isdigit() for ch in password)
has_special = any(ch in string.punctuation for ch in password)

if (len(password) >= 8 and has_upper and has_lower
      and has_digit and has_special):
   print("Strong Password")
else:
   print("Weak Password")
```

5. Sample Output

Enter password: Hello@123

Strong Password

6. Test Cases

Test Case	Input Password	Expected Output
TC1	Hello@123	Strong Password
TC2	hello123	Weak Password
TC3	Ab1@	Weak Password
TC4	Abcdefgh!	Weak Password
TC5	abcdefgh1!	Weak Password
TC6	ABCD@123	Weak Password
TC7	Welcome1!	Strong Password

7. Result

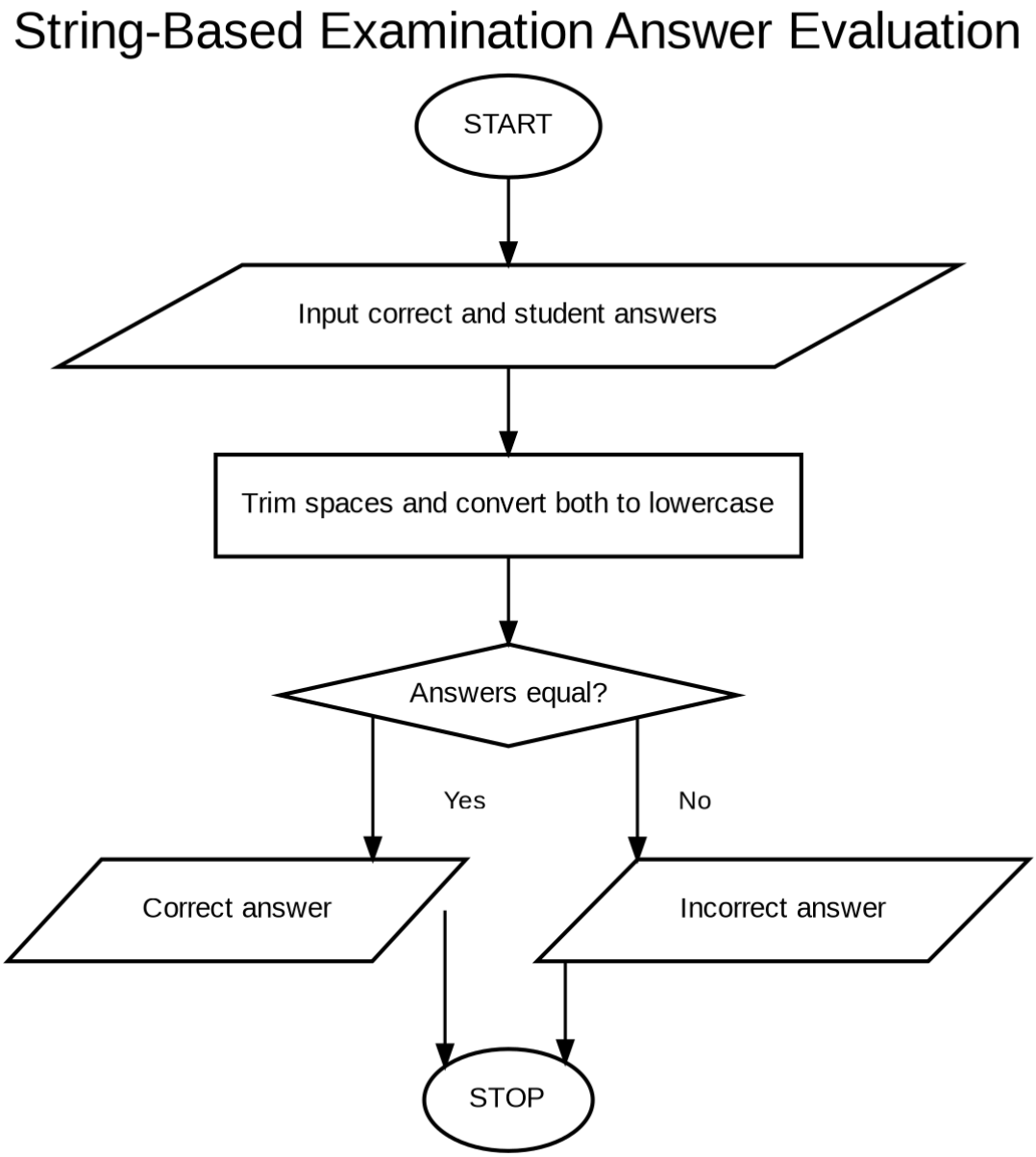
The program successfully validates password strength by checking its length, uppercase letters, lowercase letters, digits, and special characters.

PROGRAM 8: STRING-BASED ANSWER EVALUATION

1. Algorithm

1. Start.
2. Read the correct answer.
3. Read the student's answer.
4. Remove unnecessary spaces from both answers.
5. Convert both answers to lowercase.
6. Compare the correct answer with the student's answer.
7. If both answers are equal, display "Correct Answer".
8. Otherwise, display "Incorrect Answer".
9. Stop.

2. Flowchart



3. Pseudocode

BEGIN

INPUT correct_answer

INPUT student_answer

correct_answer = LOWERCASE(STRIP(correct_answer))

```
  student_answer = LOWERCASE(STRIP(student_answer))

  IF correct_answer == student_answer THEN
    PRINT "Correct Answer"
  ELSE
    PRINT "Incorrect Answer"
  END IF

END
```

4. Python Code

```
correct_answer = input("Enter correct answer: ").strip().lower()
student_answer = input("Enter student's answer: ").strip().lower()

if correct_answer == student_answer:
  print("Correct Answer")
else:
  print("Incorrect Answer")
```

5. Sample Output

```
Enter correct answer: New Delhi
Enter student's answer: new delhi
Correct Answer
```

6. Test Cases

Test Case	Correct Answer	Student Answer	Expected Output
TC1	New Delhi	new delhi	Correct Answer

TC2	Python	Java	Incorrect Answer
TC3	Hyderabad	Hyderabad	Correct Answer
TC4	Hello	hello	Correct Answer
TC5	Apple	apple	Correct Answer
TC6	Computer	Computer Science	Incorrect Answer

7. Result

The program successfully evaluates a student's answer by ignoring differences in letter case and unnecessary spaces at the beginning and end of the answer.

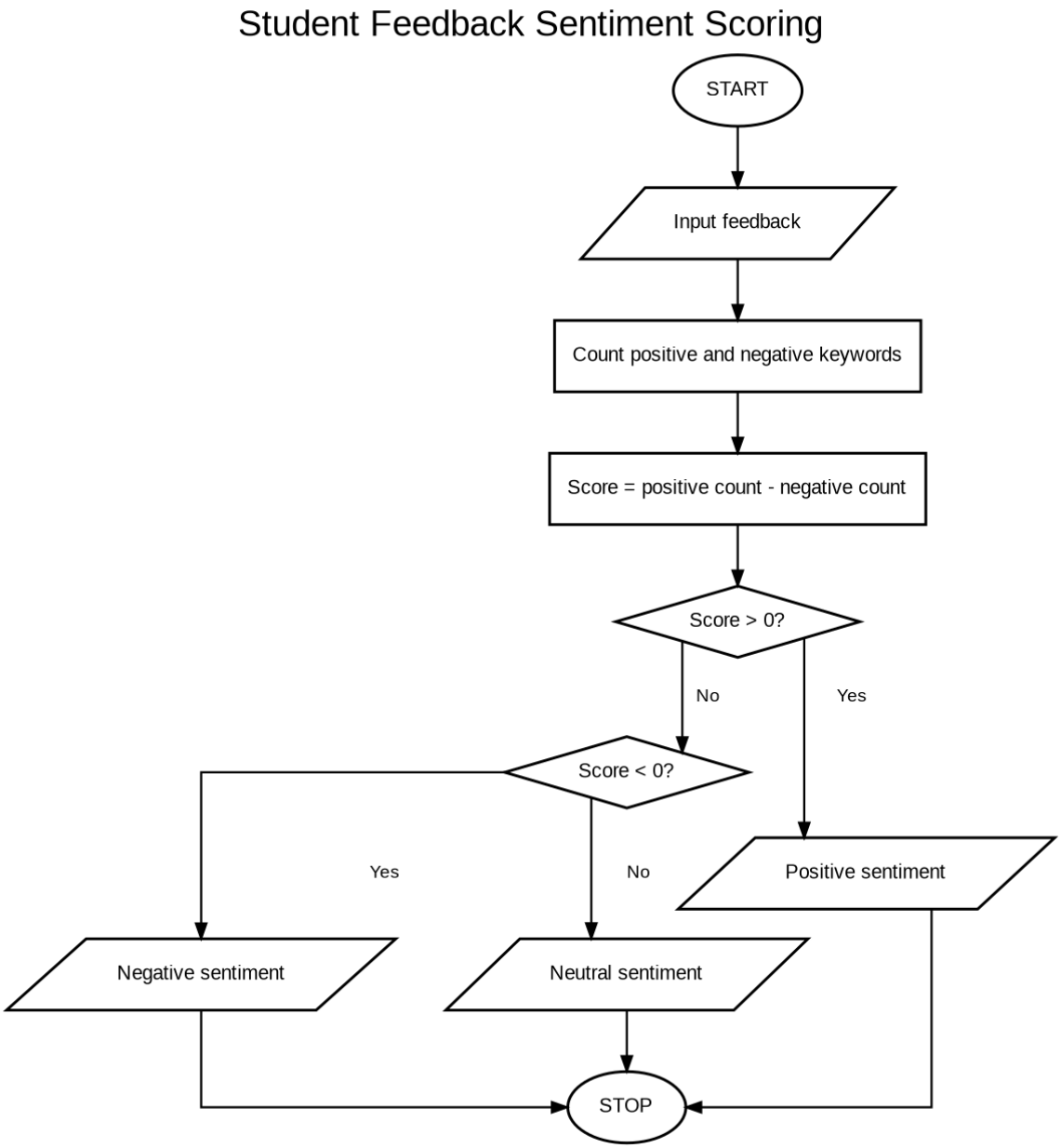
Program 9: Student Feedback Sentiment Scoring

1. Algorithm

1. Start.
2. Read feedback from the user.
3. Convert the feedback into lowercase.
4. Remove punctuation marks from the feedback.
5. Split the feedback into individual words.
6. Define positive and negative word sets.
7. Count the positive words in the feedback.
8. Count the negative words in the feedback.
9. Calculate the sentiment score by subtracting the negative count from the positive count.
10. If the score is greater than zero, display "Positive Feedback".
11. If the score is less than zero, display "Negative Feedback".
12. Otherwise, display "Neutral Feedback".
13. Display the sentiment score.

14. Stop.

2. Flowchart



3. Pseudocode

START

INPUT feedback

CONVERT feedback TO lowercase

REMOVE punctuation marks

```
    SPLIT feedback INTO words

    DEFINE positive_words
    DEFINE negative_words

    COUNT positive words
    COUNT negative words

    score = positive_count - negative_count

    IF score > 0 THEN
        DISPLAY "Positive Feedback"
    ELSE IF score < 0 THEN
        DISPLAY "Negative Feedback"
    ELSE
        DISPLAY "Neutral Feedback"
    END IF

    DISPLAY sentiment score
STOP
```

4. Python Code

```
import string

feedback = input("Enter feedback: ").lower()
feedback = feedback.translate(
```

```
    str.maketrans("", "", string.punctuation)
)
words = feedback.split()

positive_words = {
    "good", "great", "excellent",
    "helpful", "happy", "nice"
}

negative_words = {
    "bad", "poor", "worst",
    "slow", "unhappy", "boring"
}

positive_count = sum(
    1 for word in words if word in positive_words
)

negative_count = sum(
    1 for word in words if word in negative_words
)

score = positive_count - negative_count

if score > 0:
```

```
    print("Positive Feedback")
elif score < 0:
    print("Negative Feedback")
else:
    print("Neutral Feedback")

print("Sentiment Score:", score)
```

5. Sample Output

Enter feedback: The class was good and helpful

Positive Feedback

Sentiment Score: 2

6. Test Cases

Test Case	Input Feedback	Positive Count	Negative Count	Expected Output
TC1	The class was good and helpful	2	0	Positive Feedback
TC2	The service was bad and slow	0	2	Negative Feedback
TC3	The class was average	0	0	Neutral Feedback
TC4	Excellent and great service	2	0	Positive Feedback
TC5	Worst and boring experience	0	2	Negative Feedback
TC6	Hello everyone	0	0	Neutral Feedback

7. Result

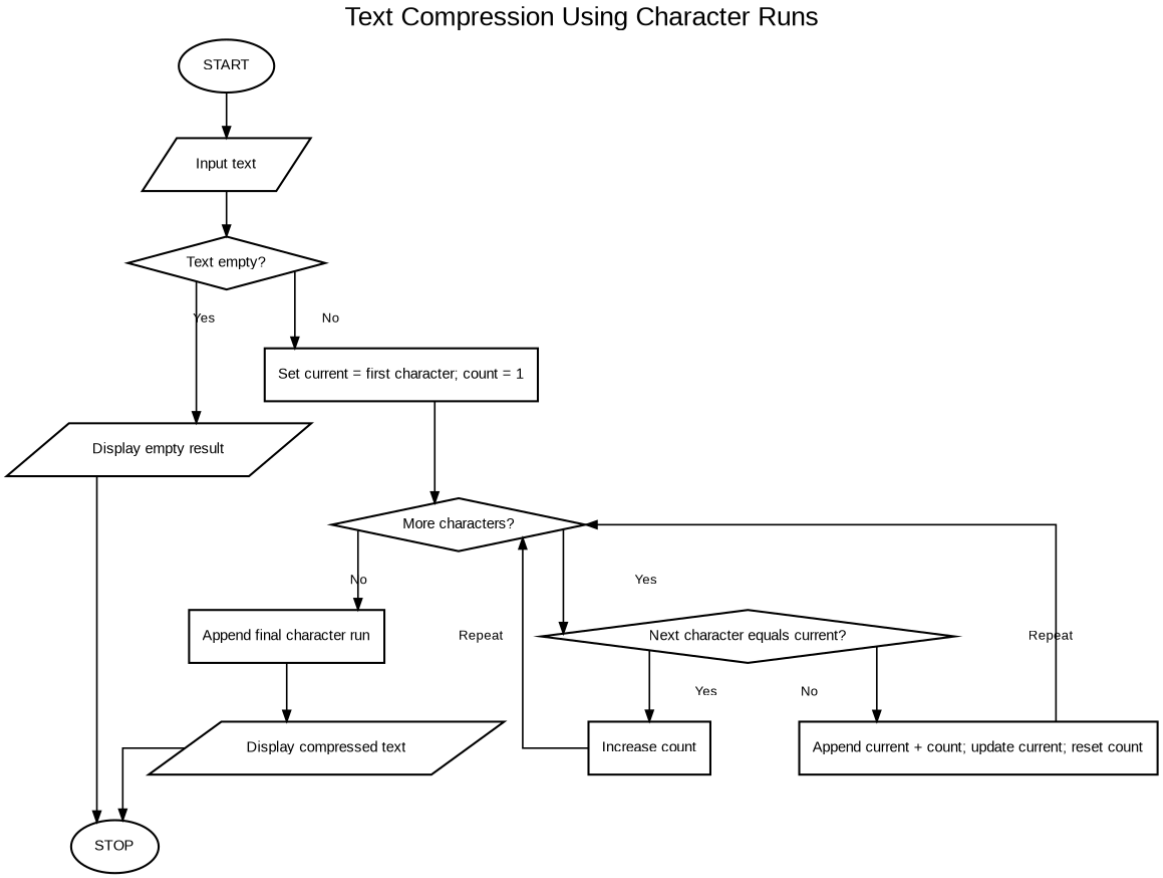
The program successfully analyzes student feedback using predefined positive and negative words, classifies the sentiment as positive, negative, or neutral, and calculates the sentiment score.

Program 10: Text Compression Using Character Runs

1. Algorithm

1. Start.
2. Read a string from the user.
3. Initialize an empty compressed string.
4. Initialize a counter to 1.
5. Traverse the string from the second character.
6. Compare the current character with the previous character.
7. If both characters are equal, increment the counter.
8. Otherwise, append the previous character and its count to the compressed string.
9. Reset the counter to 1.
10. After the loop, append the last character and its count if the string is not empty.
11. Display the compressed string.
12. Stop.

2. Flowchart



3. Pseudocode

```
START
  INPUT text
  INITIALIZE compressed = ""
  INITIALIZE count = 1

  FOR i = 1 TO length(text) - 1
    IF text[i] = text[i - 1] THEN
      count = count + 1
    ELSE
      compressed = compressed + text[i - 1] + count
      count = 1
```

```
    END IF
  END FOR

  IF length(text) > 0 THEN
    compressed = compressed + text[last_index] + count
  END IF

  DISPLAY compressed
STOP
```

4. Python Code

```
text = input("Enter a string: ")

compressed = ""
count = 1

for i in range(1, len(text)):
  if text[i] == text[i - 1]:
    count += 1
  else:
    compressed += text[i - 1] + str(count)
    count = 1

if len(text) > 0:
  compressed += text[-1] + str(count)
```

print("Compressed String:", compressed)

5. Sample Output

Enter a string: aaabbcccc

Compressed String: a3b2c4

6. Test Cases

Test Case	Input String	Expected Compressed String
TC1	aaabbcccc	a3b2c4
TC2	abc	a1b1c1
TC3	aaaa	a4
TC4	aabbcc	a2b2c2
TC5	xxxyzz	x3y1z2
TC6	aabbbba	a2b4a1

7. Result

The program successfully compresses a string by replacing consecutive repeated characters with each character followed by its repetition count.